mygit — Complete Revision Guide

Built from scratch in Java in 24 hours

Table of Contents

1. The Big Picture
2. Project Structure
3. The Object Store — Foundation of Everything
4. SHA-1 Hashing — How Git Identifies Everything
5. The Four Object Types
6. git init
7. git add — Staging Files
8. The Index — Staging Area
9. git commit — Building Snapshots
10. HEAD — The Anchor Point
11. RefStore — Managing Pointers
12. git log — Walking History
13. git status — Three-Way Comparison
14. git branch — Just a File
15. git checkout — Switching Branches
16. git diff — LCS Algorithm
17. git merge — Fast-Forward and Three-Way
18. git restore --staged — Unstaging
19. git reset — Moving HEAD Backward
20. git stash — Shelving Work
21. .gitignore — Skipping Files
22. AI Agent — Natural Language Git
23. Web Dashboard
24. Architecture Summary
25. Complete Demo Script

---

1. The Big Picture

Git is built on one core idea: **content-addressable storage.**

Every file, every directory snapshot, every commit — is stored as an object. Each object is identified by a SHA-1 hash of its content. Same content = same SHA. Different content = different SHA. Always.

```
your file → SHA-1 hash → stored at .git/objects/{sha}
```

This means:

- Git never stores the same content twice (deduplication)
- Corruption is instantly detectable (SHA mismatch)
- Any version of any file is recoverable forever

The entire history of your project is a graph of immutable objects pointing to each other. Nothing is ever modified. Nothing is ever deleted (until `git gc`). Only new objects are added.

---

2. Project Structure

```
MyGit/
├── pom.xml
└── src/
    ├── main/java/org/divesh/
    │   ├── Main.java                    ← Picocli entry point
    │   ├── core/
    │   │   ├── GitObject.java           ← sealed interface (4 types)
    │   │   ├── ObjectStore.java         ← reads/writes .git/objects/
    │   │   ├── RefStore.java            ← reads/writes HEAD + branches
    │   │   ├── Index.java               ← reads/writes .git/index
    │   │   ├── Repository.java          ← facade, ties everything together
    │   │   ├── DiffEngine.java          ← LCS algorithm
    │   │   ├── DiffLine.java            ← ADD/REMOVE/EQUAL record
    │   │   ├── MergeService.java        ← fast-forward + three-way merge
    │   │   ├── RestoreService.java      ← unstage logic
    │   │   ├── ResetService.java        ← reset logic
    │   │   └── StashService.java        ← stash logic
    │   ├── commands/
    │   │   ├── InitCommand.java
    │   │   ├── AddCommand.java
    │   │   ├── CommitCommand.java
    │   │   ├── LogCommand.java
    │   │   ├── StatusCommand.java
    │   │   ├── BranchCommand.java
    │   │   ├── CheckoutCommand.java
    │   │   ├── DiffCommand.java
```

```
    |   |   ├── MergeCommand.java
    |   |   ├── RestoreCommand.java
    |   |   ├── ResetCommand.java
    |   |   ├── StashCommand.java
    |   |   ├── HashObjectCommand.java
    |   |   ├── CatFileCommand.java
    |   |   └── AgentCommand.java
    |   ├── agent/
    |   |   ├── OpenAIClient.java        ← calls OpenAI API
    |   |   ├── CommandExecutor.java     ← runs mygit as subprocess
    |   |   ├── AgentResponse.java       ← command + explanation record
    |   |   └── GitAgent.java            ← main TUI loop
    |   └── web/
    |       └── WebServer.java           ← built-in Java HTTP server
    └── test/java/org/divesh/
        └── core/
            └── ObjectStoreTest.java
```

Build order within core:

```
GitObject → ObjectStore → RefStore → Index → Repository → Services → Commands
```

Why Picocli and not Spring Boot for CLI? Spring Boot is a web framework — it has HTTP, DI containers, bean lifecycle. For a CLI that reads/writes files, it adds slow startup and unnecessary complexity. Picocli is a lightweight annotation-based CLI library that gives you subcommands, flags, help text, and type conversion with zero overhead.

---

3. The Object Store — Foundation of Everything

`.git/objects/` is the heart of Git. Every version of every file lives here permanently.

The storage format:

```
input:   "blob 13\0hello world\n"
          ────┬─── ──────┬─────
              |          └── actual content
              └── header: type + space + size + null byte

stored:  zlib_compress(header + content)
path:    .git/objects/{sha[0:2]}/{sha[2:]}
```

Why split the path at 2 characters? Each SHA produces a 40-char hex string. The first 2 chars become the folder (256 possible folders: 00-ff). The remaining 38 become the

filename. Without this, a large repo would have hundreds of thousands of files in one directory — most filesystems get very slow at that scale.

Why zlib compress? Saves disk space. Source code compresses very well (60-80% reduction). The decompression is fast enough that it's always worth it.

Key insight: ObjectStore is the ONLY class that touches `.git/objects/`. Everything else asks it for objects by SHA.

```java
// the two core methods
String hashObject(byte[] content, String type, boolean write)
GitObject readObject(String sha)
```

---

4. SHA-1 Hashing — How Git Identifies Everything

SHA-1 is a function: any input bytes → always exactly 20 bytes output (displayed as 40 hex chars).

Four properties that make it perfect for Git:

```
1. same input    → same SHA (always, deterministic)
2. one bit change → completely different SHA (avalanche effect)
3. cannot reverse → SHA doesn't reveal the content
4. fixed size output → 40 chars regardless of file size
```

What Git actually hashes (not just the file):

```
SHA1("blob 13\0hello world\n")     ← header prepended first
      ──────── ──────────────
      header    content
```

The header includes the type so a blob and a commit with identical content produce different SHAs and don't collide in the objects folder.

In Java — zero external dependencies:

```java
```

```
MessageDigest md = MessageDigest.getInstance("SHA-1");
byte[] rawHash = md.digest(full);  // 20 bytes

// convert to 40 hex chars
StringBuilder sha = new StringBuilder();
for (byte b : rawHash) {
    sha.append(String.format("%02x", b));
}
// %02x: 02 = pad to 2 chars, x = hex lowercase
```

Common question: why does `write=false` exist in hashObject? `git status` needs to hash files to compare them against the index — but it doesn't want to store anything. `write=false` computes the SHA without touching disk. `write=true` (used by `add`) stores the blob. Same hashing logic, different outcome.

Verification: `git hash-object file.txt` and `mygit hash-object file.txt` on the same file must produce identical SHAs. If they don't, the header format is wrong (most common bug: wrong arraycopy order — header must come first).

---

5. The Four Object Types

`GitObject.java` — sealed interface with four records:

java

```
public sealed interface GitObject
    permits GitObject.Blob, GitObject.Tree,
            GitObject.Commit, GitObject.Tag {
    String type();
}
```

Why sealed? The compiler knows there are exactly four types. Exhaustive switch statements — no default case needed, and the compiler tells you if you miss a case.

Why records? Records are immutable data containers. Git objects should never change after creation — immutability is enforced automatically. Records also auto-generate constructor, getters, equals, hashCode, toString.

Blob

Stores raw file content. No filename, no metadata. Just bytes.

```
header:  "blob 13\0"
content: raw file bytes
```

Two files with identical content share one blob. The filename lives in the tree, not the blob.

Tree

Stores a directory listing. Maps filenames to blob/tree SHAs.

```
format (binary per entry):
"100644 Hello.java\0" + [20 raw SHA bytes]
"040000 src\0"        + [20 raw SHA bytes of subtree]

modes:
100644 = regular file
040000 = directory (points to another tree)
```

Why binary for SHA in trees (not hex)? A tree with 100 files needs 100×40=4000 bytes if hex, but only 100×20=2000 bytes if binary. Git chose compact binary for trees since repos can have thousands of files.

Commit

Stores a snapshot with metadata. Plain text.

```
tree d8329fc...
parent a1b2c3...      ← omitted for first commit
                      ← two parent lines for merge commits
author Divesh <d@email.com> 1711000000 +0530
committer Divesh <d@email.com> 1711000000 +0530

commit message here
```

The blank line between headers and message is mandatory. `parseCommit` uses it to know when the message starts.

Tag

Named pointer to a commit. Used for releases.

```
object a1b2c3...
type commit
tag v1.0
tagger Divesh <d@email.com> 1711000000 +0530

release message
```

---

6. git init

Creates the `.git/` directory structure. That's the entire implementation.

```java
public static void init(Path directory) throws IOException {
    Path gitDir = directory.resolve(".git");

    if (Files.exists(gitDir)) {
        // already initialized — print reinit message
        System.out.println("Reinitialized existing repository...");
        return;
    }

    Files.createDirectories(gitDir.resolve("objects"));
    Files.createDirectories(gitDir.resolve("refs").resolve("heads"));
    Files.writeString(gitDir.resolve("HEAD"),
            "ref: refs/heads/main\n");

    System.out.println("Initialized empty git repository in "
            + gitDir.toAbsolutePath());
}
```

Why `createDirectories` not `createDirectory`? `createDirectories` creates the full chain (`.git/refs/heads/` in one call) and doesn't throw if it already exists. `createDirectory` would fail on the second call.

Why static? Init creates the repo — the repo doesn't exist yet. You can't create a Repository instance before `.git/` exists. Static factory method is the correct pattern here.

After init:

```
.git/
├── HEAD          → "ref: refs/heads/main\n"
├── objects/      → empty
└── refs/heads/   → empty
```

Your actual files are completely untouched. Git just watches from the sidelines.

---

7. git add — Staging Files

`git add Hello.java` does exactly five things:

```
1. read Hello.java bytes from disk
2. build header: "blob {byteCount}\0"
3. SHA1(header + content) → 40 char hex
4. zlib compress (header + content)
5. write compressed bytes to .git/objects/{sha[0:2]}/{sha[2:]}
6. write "{sha}  Hello.java" into .git/index
```

The blob is stored permanently after add. Not after commit — after add. Commit just wraps existing blobs with metadata.

`git add .` — adding all files:

java

```java
Files.walk(directory)
     .filter(Files::isRegularFile)
     .filter(p -> !p.startsWith(absoluteRoot.resolve(".git")))
     .filter(p -> !p.getFileName().toString().startsWith("."))
     .filter(p -> !gitIgnore.isIgnored(p))
     .forEach(p -> add(p));
```

Common question: if SHA can't be reversed, how does Git recover files? Git never needs to reverse the SHA. It uses the SHA as an address — reads the file at `.git/objects/{sha}`, decompresses it, strips the header, and returns the content. The SHA is a lookup key, not an encryption.

Deduplication: Two files with identical content produce the same SHA. When the second blob would be written, the file already exists at that path. Git skips writing it. One object on disk, two index entries pointing to it.

Relativize paths: The index stores paths relative to repo root, not absolute paths. `repoRoot.relativize(absoluteFilePath)` strips the prefix. Without this, the repo would break the moment it's moved to a different folder.

---

8. The Index — Staging Area

The index is the middle layer between your working directory and the object store.

```
working directory → add → INDEX → commit → object store
```

Implementation: A simple text file at `.git/index`. One line per entry:

```
8ab686eafeb1f44702738c8b0f24f2567c36da6d  Hello.java
3fc291aa4567293837489123456789abc123456  src/Utils.java
```

Why `LinkedHashMap` not `HashMap`? Preserves insertion order. Files stay in the order they were added. Important for consistent tree building.

Why split on two spaces? `line.split("  ", 2)` — the `2` limits splits to at most 2 parts. This handles filenames that contain spaces (e.g., `my file.java` wouldn't be split incorrectly).

Four methods:

```
load()          →  read .git/index → Map<path, sha>
save(map)       →  Map<path, sha>  → write .git/index
add(path, sha)→  load + put + save
getEntries()   →  convenience wrapper for load()
```

Pattern: load → modify in memory → save back. Java can only read or write an entire file, not edit individual lines.

---

9. git commit — Building Snapshots

Commit does four things:

Step 1 — Build tree from index

`writeTreeRecursive` converts the flat index map into nested tree objects:

```
index (flat):                    trees (nested):
README.md → sha1                 root tree:
src/Hello.java → sha2              README.md → blob sha1
src/Utils.java → sha3              src/      → tree sha_src

                                 src tree:
                                   Hello.java → blob sha2
                                   Utils.java → blob sha3
```

Why recursive? The index stores paths like `src/Hello.java` but Git trees are nested. The recursive function splits entries by directory prefix, builds subtrees bottom-up (deepest first), and the root tree last.

Why sort alphabetically? Git always sorts tree entries by filename. Same files in different order = different SHA. Without sorting, your tree SHA won't match real Git's.

Step 2 — Build commit object

```
tree {treeSha}\n
parent {parentSha}\n     ← omitted for first commit
author {author} {timestamp} +0530\n
committer {author} {timestamp} +0530\n
\n
{message}\n
```

The blank line is mandatory — `parseCommit` uses it as the separator between structured headers and the free-form message.

Step 3 — Store commit object Same `hashObject` call as blobs, but type is `"commit"`.

Step 4 — Update branch pointer

```
refStore.updateHead(commitSha)
→ reads HEAD → "ref: refs/heads/main"
→ writes commitSha into .git/refs/heads/main
```

First commit vs subsequent commits:

```
first commit:   parentSha = null  → no parent line
second commit:  parentSha = first commit SHA → parent line added
merge commit:   two parent lines
```

---

10. HEAD — The Anchor Point

HEAD is a single text file at `.git/HEAD`. It answers one question at all times: **"where am I right now?"**

Two states:

```
Attached (normal):
.git/HEAD = "ref: refs/heads/main"
→ you are on the main branch
→ HEAD points to branch, branch points to commit

Detached:
.git/HEAD = "a1b2c3..."
→ you checked out a specific commit directly
→ new commits have nothing pointing to them (dangerous)
```

The two-hop chain:

```
HEAD → "ref: refs/heads/main"
     → .git/refs/heads/main → "a1b2c3..."
     → commit a1b2c3
```

HEAD's role in every operation:

```
commit    → which branch file to update (branch moves, HEAD stays)
log       → where to start walking history
status    → what was last committed (baseline for comparison)
checkout  → HEAD file itself changes to new branch
reset     → move backwards from HEAD's current position
merge     → HEAD's branch = "our side"
stash     → save diff from HEAD, restore to HEAD
diff HEAD → HEAD is the baseline
```

Common question: why doesn't HEAD move on commit? HEAD always points to the current branch. The branch pointer moves forward to the new commit. HEAD doesn't need to change — it still correctly points to the branch which now points to the new commit.

---

11. RefStore — Managing Pointers

RefStore is the only class that touches `.git/HEAD` and `.git/refs/`. Just like ObjectStore owns objects, RefStore owns pointers.

Five methods:

java

```
readHead()          // reads .git/HEAD as-is
resolveHead()       // follows HEAD → branch → commit SHA
                    // returns null if no commits yet
updateHead(sha)     // writes sha to whatever HEAD points at
createBranch(name, sha)  // writes .git/refs/heads/{name}
getCurrentBranch() // strips "ref: refs/heads/" → branch name
```

`resolveHead()` is null-safe: On a fresh repo after init, `.git/refs/heads/main` doesn't exist yet (no commits). `resolveHead()` returns null. `commit()` checks for null to know it's the first commit and skips the parent line.

---

12. git log — Walking History

The simplest command — just follows the parent chain:

Why most recent first? You start at HEAD (latest) and walk backwards. The chain points backwards in time — each commit knows its parent, not its children.

`--oneline` flag:

```java
System.out.println(sha.substring(0, 7) + " " + commit.message());
```

First 7 chars of SHA is standard shorthand. Collision probability is astronomically low for typical repo sizes.

---

13. git status — Three-Way Comparison

Status compares three things:

```
working directory  ↔  index  ↔  last commit
      (disk)          (staged)    (committed)
```

Comparison 1: index vs last commit

```
for each file in index:
    not in last commit  →  "new file"     (staged, never committed)
    sha differs         →  "modified"     (staged change)
```

Comparison 2: disk vs index

```
walk every file on disk:
    not in index        →  "untracked"
    disk sha ≠ index sha →  "modified not staged"
```

Key insight: SHA comparison is instant. Git doesn't read file content to detect changes — it just hashes the disk file (write=false) and compares to the index SHA. If they match, file is unchanged. If they differ, something changed.

`getCommittedFiles(commitSha)` — used by status to get the last committed state:

```
read commit → get treeSha
recursively walk tree → build flat map of filepath → sha
```

This same helper is reused in diff, merge, reset, restore.

---

14. git branch — Just a File

The most important insight about branches: A branch is literally a text file containing one SHA.

```
.git/refs/heads/main     →  "a1b2c3...\n"
.git/refs/heads/feature  →  "b9d3a2...\n"
```

Three operations:

```
create:  write current HEAD SHA into new file
list:    list all files in .git/refs/heads/
delete:  delete the file
```

Why can't you delete current branch? HEAD points to it. If you deleted it, HEAD would be pointing to a non-existent file. Check `name.equals(refStore.getCurrentBranch())` before deleting.

Bug that tripped us up: `listBranches()` used `Files.list()` which returns a stream that must be closed. Wrapping in `try (var branches = Files.list(headsDir))` fixes the resource leak that caused `error: null`.

---

15. git checkout — Switching Branches

Four things happen in sequence:

```
1. resolve target branch → get its commit SHA
2. get current files (from index)
3. get target files (from target commit's tree)
4. delete files that exist now but NOT in target
5. delete empty parent directories
6. clear index
7. restore target files to disk
8. rebuild index from target tree
9. write "ref: refs/heads/{branch}" into .git/HEAD
```

The bug we found and fixed: Initially checkout only wrote new files but didn't delete files from the old branch. `feature-src/Feature.java` would stay on disk after switching to main because main's tree doesn't mention it. Fix: compare current index vs target files, delete anything not in target.

Empty directory cleanup:

```java
if (list.findAny().isEmpty()) {
    Files.deleteIfExists(parent);
}
```

After deleting `feature-src/Feature.java`, the `feature-src/` directory is empty. Clean it up.

`-b` **flag:** Creates the branch first, then calls checkout. Two operations chained.

Safety check (not implemented, post-hackathon): Real Git refuses checkout if you have uncommitted changes that would be overwritten. Check disk SHA vs index SHA for each file before proceeding.

---

16. git diff — LCS Algorithm

Diff shows which lines changed between two versions of a file.

Three modes:

```
diff          →  working directory vs index
diff --cached →  index vs last commit
diff b1..b2   →  compare two branches
```

LCS (Longest Common Subsequence) algorithm:

Find the longest sequence of lines that appears in both versions in the same order. Then:

```
lines in LCS          → unchanged (space prefix)
lines only in old     → removed   (- prefix)
lines only in new     → added     (+ prefix)
```

The DP table:

```
old = [A, B, C, D]
new = [A, C, D, E]

     ""  A  C  D  E
""  [ 0  0  0  0  0 ]
A   [ 0  1  1  1  1 ]
B   [ 0  1  1  1  1 ]
C   [ 0  1  2  2  2 ]
D   [ 0  1  2  3  3 ]

Fill rule:
if old[i] == new[j]: dp[i][j] = dp[i-1][j-1] + 1
else:                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

Backtrack → LCS = [A, C, D]
diff output:
 A   (unchanged)
-B   (removed)
 C   (unchanged)
 D   (unchanged)
+E   (added)
```

Output format:

```
--- src/Hello.java    ← old file
+++ src/Hello.java    ← new file
@@ -1 +1 @@           ← line numbers
 public class Hello {}
+// new line
```

`writeTree` **recursive — how subdirectories work in diff:** `collectFlatEntries` recursively walks a tree and builds a flat map `{filepath → sha}` for comparison. Used in diff, merge, status, reset, restore — same pattern everywhere.

---

17. git merge — Fast-Forward and Three-Way

Two completely different paths:

Fast-Forward

Happens when one branch is directly ahead of the other — no divergence.

```
main:    A → B → C
feature:         C → D → E

walk feature ancestors: E → D → C ← found main's commit!
fast-forward possible → just move main pointer to E
no new commit needed, history stays linear
```

Detection:

java

```java
String sha = targetSha;
while (sha != null) {
    if (sha.equals(currentSha)) return true;  // FF possible
    sha = commit.parentSha();
}
return false;  // not FF, need three-way
```

Three-Way Merge

Happens when both branches have diverged — both have commits the other doesn't.

Why three versions? Without the ancestor you can't tell who changed what. With ancestor:

```
ancestor: Hello.java has 3 lines
main:     Hello.java has 3 lines  → main didn't change it
feature:  Hello.java has 4 lines  → feature added a line
→ take feature's version, no conflict
```

Finding common ancestor:

java

```java
// collect ALL ancestors of main into Set
Set<String> ancestors = new HashSet<>();
// walk sha1 chain → add each to set

// walk sha2 chain → first one in set = common ancestor
```

Six cases for each file:

```
1. same SHA in ours and theirs    → keep (unchanged or both same change)
2. null in ours, exists in theirs → add it (new file from feature)
3. exists in ours, null in theirs → keep (our new file)
4. ours == ancestor, theirs ≠     → take theirs (only they changed it)
5. theirs == ancestor, ours ≠     → keep ours (only we changed it)
6. both ≠ ancestor, both differ   → CONFLICT
```

Conflict markers:

```
<<<<<<< main
our version of the file
=======
their version of the file
>>>>>>> feature
```

Merge commit has two parents:

```
tree    merged_tree_sha
parent  our_commit_sha      ← main's tip
parent  their_commit_sha    ← feature's tip
```

`MERGE_HEAD` (post-hackathon improvement): Real Git writes `.git/MERGE_HEAD` during a conflict so the next `commit` automatically creates a two-parent merge commit. We skipped this for the hackathon — the manual commit after conflict resolution creates a normal single-parent commit instead.

---

18. git restore --staged — Unstaging

Undoes `git add` without touching the file on disk.

Three scenarios:

```
Scenario 1 — file was committed before, then modified and staged:
    index: Hello.java → sha_new
    after restore: index: Hello.java → sha_old (committed version)
    disk: unchanged

Scenario 2 — brand new file never committed, just staged:
    index: NewFile.java → sha_new
    after restore: NewFile.java REMOVED from index
    disk: unchanged (now "untracked")

Scenario 3 — no commits at all:
    just remove from index entirely
```

Logic:

java

```java
String commitSha = refStore.resolveHead();
if (commitSha == null) {
    index.remove(filepath);  // no commits → just remove
    return;
}
Map<String, String> committed = getCommittedFiles(commitSha);
if (committed.containsKey(filepath)) {
    index.add(filepath, committed.get(filepath));  // restore to committed SHA
} else {
    index.remove(filepath);  // never committed → remove entirely
}
```

---

19. git reset — Moving HEAD Backward

Three modes — each more destructive than the last:

```
              HEAD pointer   index          disk files
--soft        moved back     unchanged      unchanged
--mixed       moved back     rebuilt        unchanged     ← default
--hard        moved back     rebuilt        rebuilt
```

Resolving `HEAD~1`:

java

```
if (target.startsWith("HEAD~")) {
    int steps = Integer.parseInt(target.substring(5));
    String sha = refStore.resolveHead();
    for (int i = 0; i < steps; i++) {
        sha = commit.parentSha();  // walk back N steps
    }
    return sha;
}
```

`rebuildIndex(commitSha)`:

java

```
Map<String, String> files = getCommittedFiles(commitSha);
index.clear();
files.forEach((path, sha) -> index.add(path, sha));
```

`--hard` extra step: After rebuilding index, also write files to disk. Delete files not in target, restore files that are in target.

Use cases:

```
--soft:  "I want to redo this commit with different message"
         changes stay staged, just re-commit

--mixed: "I want to unstage everything and rethink"
         changes still on disk, nothing staged

--hard:  "Discard everything, go back to that commit"
         nuclear option — data lost
```

---

20. git stash — Shelving Work

Stash is secretly just a commit. A special commit that doesn't appear in normal history.

`git stash` — saving:

```
1. hash all current disk files → blobs (capture WIP state)
2. build tree from current files
3. create commit:
       tree    = WIP tree
       parent  = HEAD      ← "I was saved while on this commit"
       message = "WIP on main: a1b2c3 last commit msg"
4. write commit SHA to .git/refs/stash
5. reset working directory to HEAD (hard reset)
```

Why does stash commit need a parent (HEAD)? When you `stash pop`, Git needs to know what the repo looked like before the stash — so it can restore correctly. The parent tells it "these WIP changes were on top of THIS commit." Without it, Git wouldn't know which files belonged to the repo baseline vs which were stash-specific.

`git stash pop` — restoring:

```
1. read .git/refs/stash → stash commit SHA
2. read stash commit → get its tree
3. restore stash files to disk
4. update index
5. delete .git/refs/stash
```

Why doesn't stash appear in `git log`? `log` walks from HEAD → branch → commit chain. The stash commit is only reachable via `.git/refs/stash` — which log never reads. It's an orphaned commit in the object store, invisible to normal history traversal.

---

21. .gitignore — Skipping Files

Applied during `addAll()` as a filter:

java

```java
Files.walk(directory)
     .filter(p -> !gitIgnore.isIgnored(p))  // ← this filter
     .forEach(p -> add(p));
```

Pattern matching:

```
*.class   →  filename ends with .class (glob)
target/   →  path contains this directory name
secret.txt → exact filename match
```

Java `PathMatcher` with glob syntax:

Important: `.gitignore` only affects `add` — it doesn't affect already-tracked files. If `Hello.class` was committed before you added `*.class` to `.gitignore`, it stays tracked. `git rm --cached Hello.class` would be needed (not implemented).

---

22. AI Agent — Natural Language Git

The flow:

```
user types: "undo my last commit but keep the files"
    ↓
OpenAI API called with system prompt
    ↓
response:
    COMMAND: mygit reset --soft HEAD~1
    EXPLAIN: moves HEAD back one commit, keeps changes staged
    ↓
show command + explanation
if dangerous command → show ⚠ WARNING
ask y/n confirmation
    ↓
if y → CommandExecutor runs it as subprocess → show output
if n → cancelled
```

Why confirmation before execution? Safety. A misunderstood command like `mygit reset --hard HEAD` destroys uncommitted work. The user sees what will happen before it happens. Dangerous commands (containing `--hard`, `--mixed`, `branch -d`) show an extra warning.

System prompt key rules:

```
respond in EXACTLY this format:
COMMAND: <the mygit command>
EXPLAIN: <one line, max 10 words>

if cannot translate:
COMMAND: UNKNOWN
EXPLAIN: could not understand the request
```

Manual JSON parsing (no Gson needed):

```java
// find "content": "..." in response
String marker = "\"content\": \"";
int start = json.indexOf(marker) + marker.length();
// skip escaped chars properly, find closing quote
```

Common bug we fixed: OpenAI's JSON response has `"content": "` (with space after colon) but naive parsing searched for `"content":"` (no space). The fix tries both patterns.

`CommandExecutor`:

```java
// "mygit reset --soft HEAD~1"
// → ["java", "-jar", "MyGit.jar", "reset", "--soft", "HEAD~1"]
ProcessBuilder pb = new ProcessBuilder(cmdList);
pb.directory(new File(workingDir));
pb.redirectErrorStream(true);  // merge stdout + stderr
```

Why `redirectErrorStream(true)`? Programs output to two streams: stdout (normal output) and stderr (error messages). If you only read stdout, error messages are lost. Merging them into one stream ensures you see everything.

State machine:

```
WAITING_FOR_QUERY   → user types a request
WAITING_FOR_CONFIRM → showing command, waiting y/n
```

---

23. Web Dashboard

Built with Java's built-in `HttpServer` — zero extra dependencies:

```java
HttpServer server = HttpServer.create(
        new InetSocketAddress(8080), 0);
server.createContext("/", new IndexHandler());    // serves HTML
server.createContext("/run", new RunHandler());  // executes commands
server.start();
```

Two endpoints:

```
GET  /      → serve the HTML dashboard
POST /run   → receive {"command": "mygit status"}
              run via CommandExecutor
              return {"output": "On branch main..."}
```

The HTML is served as a Java string literal using text blocks (Java 15+):

```java
private String buildHtml() {
    return """
    <!DOCTYPE html>
    ...
    """;
}
```

Why not Spring Boot? We tried Spring Boot — it requires servlet container detection, proper classpath setup, and caused `ServletWebServerFactory bean not found` errors due to shade plugin conflicts. Java's built-in `HttpServer` is simpler, zero config, always works.

Frontend: Pure vanilla HTML/CSS/JS. Sidebar with quick-access buttons, scrollable output panel, input bar at bottom. Each button calls `fetch('/run', {method:'POST', body: JSON.stringify({command: 'mygit status'})})`.

---

24. Architecture Summary

The full object graph after a complete workflow:

```
HEAD
  └── "ref: refs/heads/main"
        └── .git/refs/heads/main → "b9d3a2..."
              └── commit b9d3a2  (merge commit)
                    ├── parent a1b2c3  (main's last commit)
                    ├── parent c4d5e6  (feature's last commit)
                    └── tree e7f891
                          ├── README.md    → blob sha1
                          └── src/         → tree sha2
                                ├── Hello.java → blob sha3
                                └── Utils.java → blob sha4
```

The three primitives everything is built on:

```
SHA     = identity   (same content = same SHA, always)
blob    = content    (what a file contains)
pointer = reference  (HEAD → branch → commit → tree → blobs)
```

Class responsibilities:

```
ObjectStore   → owns .git/objects/     (reads/writes all objects)
RefStore      → owns .git/HEAD         (current position)
              → owns .git/refs/        (branch pointers)
Index         → owns .git/index        (staging area)
Repository    → facade (ties all three together, commands talk to this)
MergeService  → merge logic (fast-forward + three-way + conflict)
ResetService  → reset logic (soft/mixed/hard)
RestoreService→ unstage logic
StashService  → stash logic
DiffEngine    → LCS algorithm
WebServer     → HTTP server
GitAgent      → AI TUI loop
OpenAIClient  → OpenAI API calls
CommandExecutor→ runs mygit as subprocess
```

Key design decisions:

```
Pure Java core (no Spring in core)    → fast startup, no overhead
Sealed classes for GitObject          → exhaustive switch, compiler safety
Records for data types                → immutable, auto-generated methods
Service classes for complex features  → keep Repository manageable
write=false in hashObject             → compute SHA without storing
relativize paths in index             → repo works when moved
LinkedHashMap for index               → preserves insertion order
```

---

25. Complete Demo Script

bash

```
# SETUP
cd ~/Desktop && rm -rf gitdemo && mkdir gitdemo && cd gitdemo

# INIT
mygit init
ls .git/
cat .git/HEAD

# CREATE FILES
mkdir src
echo "public class Hello { public static void main(String[] args) { System.out.
echo "public class Utils { public static String greet(String name) { return \"H
echo "# GitDemo Project" > README.md
printf "*.class\n*.log\ntarget/" > .gitignore

# BEFORE STAGING
mygit status
mygit hash-object src/Hello.java

# STAGING
mygit add .
cat .git/index
find .git/objects -type f
mygit status

# RESTORE STAGED
mygit restore --staged README.md
mygit status
mygit add README.md

# FIRST COMMIT + VERIFY CHAIN
mygit commit -m "initial commit" -a "Divesh <d@email.com>"
cat .git/refs/heads/main
mygit cat-file -p $(cat .git/refs/heads/main)
mygit log --oneline

# DIFF
echo "// updated" >> src/Hello.java
mygit diff
mygit add src/Hello.java
mygit diff --cached
mygit commit -m "add comment" -a "Divesh <d@email.com>"

# BRANCHING
mygit branch
mygit branch feature
```

```
cat .git/refs/heads/feature
mygit checkout feature
cat .git/HEAD

# FEATURE WORK
mkdir feature-src
echo "public class Payment {}" > feature-src/Payment.java
echo "// feature change" >> src/Hello.java
mygit add .
mygit commit -m "add payment" -a "Divesh <d@email.com>"

# BACK TO MAIN
mygit checkout main
ls
cat src/Hello.java
echo "// main update" >> src/Utils.java
mygit add src/Utils.java
mygit commit -m "update Utils" -a "Divesh <d@email.com>"

# DIFF BRANCHES
mygit diff main..feature

# FAST FORWARD
mygit branch hotfix
mygit checkout hotfix
echo "# fix" >> README.md
mygit add README.md && mygit commit -m "hotfix" -a "Divesh <d@email.com>"
mygit checkout main
mygit merge hotfix
mygit log --oneline

# CONFLICT
echo "// main Hello" >> src/Hello.java
mygit add src/Hello.java && mygit commit -m "main Hello" -a "Divesh <d@email.co
mygit checkout feature
echo "// feature Hello" >> src/Hello.java
mygit add src/Hello.java && mygit commit -m "feature Hello" -a "Divesh <d@email
mygit checkout main
mygit merge feature
cat src/Hello.java

# RESOLVE
echo "public class Hello { public static void main(String[] args) { System.out.
mygit add src/Hello.java
mygit commit -m "resolve conflict" -a "Divesh <d@email.com>"
mygit log --oneline
```

```
# RESET
mygit reset --soft HEAD~1
mygit status
mygit commit -m "resolve conflict" -a "Divesh <d@email.com>"
mygit reset --mixed HEAD~1
mygit status
mygit reset --hard HEAD
mygit status

# STASH
echo "// WIP" >> src/Utils.java
mygit stash
mygit status
cat src/Utils.java
mygit stash list
mygit stash pop
cat src/Utils.java

# GITIGNORE
echo "compiled" > Hello.class
echo "error" > app.log
mygit add .
mygit status

# WEB UI
mygit --web
# open http://localhost:8080

# AI AGENT (new terminal)
export OPENAI_API_KEY=sk-...
mygit ai
# show me the status
# create a branch called demo
# discard all my changes
# exit
```

Quick Reference — What Each File Owns

File	Owns	Never Touches
ObjectStore	.git/objects/	HEAD, index, refs
RefStore	.git/HEAD, .git/refs/	objects, index
Index	.git/index	objects, HEAD, refs

File	Owns	Never Touches
Repository	nothing directly	delegates to above three
MergeService	orchestrates all three	has own copies of helpers
WebServer	HTTP port 8080	git internals directly
GitAgent	terminal I/O	git internals directly

Common Bugs and Fixes

Bug	Cause	Fix
SHA doesn't match real Git	arraycopy order wrong (content before header)	header must come first
`error: null` on branch list	missing `return` after `listBranches()`	add return after listing
Files stay after checkout	restoreTree didn't delete old files	compare index vs target, delete missing
`.DS_Store` getting tracked	addAll didn't filter hidden files	`.filter(p -> !p.getFileName().toString().startsWith("."))`
Flat tree (no subdirs)	add() not using absolute paths for relativize	use `toAbsolutePath()` before relativize
OpenAI returns UNKNOWN	`extractContent()` searched for `"content":"` but JSON has `"content": "`	try both patterns with/without space
Spring Boot web fails	shade plugin conflict with boot plugin	use built-in Java HttpServer instead

Built in 24 hours at Hacksplosion 2026. Every line of code written from scratch.